Bharatiya Vidya Bhavan's
SARDAR PATEL INSTITUTE OF TECHNOLOGY
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri (W), Mumbai – 400 058.
Department of Computer Engineering

Lab Assignment

Title: Comparative Analysis of Asymmetric Cryptographic Algorithms

1. Aim

To implement and compare asymmetric cryptographic algorithms based on their mathematical foundations, key-generation process, encryption/decryption mechanism, key-exchange capability, security assumptions, and practical applications.

2. Learning Outcomes

After completing this experiment, students will be able to:

- Implement basic asymmetric cryptographic algorithms.
- Generate public and private keys.
- Perform encryption and decryption using asymmetric algorithms.
- Perform key exchange using Diffie–Hellman.
- Compare asymmetric algorithms based on their underlying mathematical problems.
- Analyze the relationship between key size and security.
- Identify suitable applications for different asymmetric cryptographic techniques.

3. Algorithms to be Implemented

Students shall implement:

1. RSA
2. ElGamal Encryption
3. Diffie–Hellman Key Exchange
4. Elliptic Curve Cryptography (ECC)

4. Dataset

Each student will be provided with:

```
asymmetric_cipher_dataset.xlsx
```

The dataset will contain approximately **100 records** consisting of plaintext messages and/or numerical message values appropriate for the algorithms.

Students shall populate the relevant columns with:

- Generated parameters
- Public keys
- Private keys
- Ciphertext
- Decrypted message
- Shared secret, where applicable
- Observations

The same dataset will be used by all students.

5. UID-Based Parameter Generation

Each student shall use their **10-digit UID** to generate their parameters.

Example:

```
UID = 2024300020
```

The UID shall be used as the seed for the random number generator:

Python

```
random.seed(int(UID))
```

This ensures that:

- The same UID always produces the same parameters.
- Different students obtain different parameters.
- The instructor can reproduce a student's parameters for verification.

5.1 RSA Parameter Generation

Using the UID as the random seed:

1. Generate two suitable prime numbers p and q.
2. Calculate:
$$n = pq \ \phi(n) = (p-1)(q-1)$$
3. Select e such that:
$$1 < e < \phi(n)$$
and
$$gcd(e, \phi(n)) = 1$$
4. Calculate the private exponent d:
$$ed \equiv 1 \pmod{\phi(n)}$$

The resulting keys are:

Public Key:

$$(e, n)$$

Private Key:

$$(d, n)$$

Students shall use these parameters to perform RSA encryption and decryption.

5.2 ElGamal Parameter Generation

Using the UID as the random seed, generate:

- A prime number p
- A generator g
- A private key x

Calculate the public component:

$$y = g^x \bmod p$$

Therefore:

Private Key:

$$x$$

Public Key:

$$(p, g, y)$$

For encryption, generate an ephemeral random value k for each message.
Students shall record the required ciphertext components and verify the decrypted message.

5.3 Diffie–Hellman Parameter Generation

Generate common public parameters:

$$p, \quad g$$

Using the UID-derived random sequence, generate private values for Alice and Bob:

$$a, \quad b$$

Calculate:

Alice's public value

$$A = g^a \bmod p$$

Bob's public value

$$B = g^b \bmod p$$

Calculate the shared secrets:
$K_A = B^a \bmod p$ $K_B = A^b \bmod p$
Verify:

$$K_A = K_B$$

5.4 ECC Parameter Generation

Use a specified elliptic curve for the experiment.
Students shall generate a private key d using the UID-derived random sequence.
Calculate the corresponding public key:

$$Q = dG$$

where:

- G = base point
- d = private key
- Q = public key

Students shall perform the required ECC point operations and verify the resulting shared value/ciphertext according to the implementation specified by the instructor.

6. Experimental Procedure

Part A – RSA

For each assigned message:

1. Generate RSA parameters.
2. Record $p, q, n, \phi(n), e, d$.
3. Encrypt the message.
4. Record the ciphertext.
5. Decrypt the ciphertext.
6. Verify that the original message is recovered.

Populate:

Message	p	q	n	e	d	Ciphertext	Decrypted Message

Part B – ElGamal

For each assigned message:

1. Generate p, g, x.
2. Calculate y.
3. Generate the ephemeral value k.
4. Encrypt the message.
5. Record the ciphertext.
6. Decrypt the ciphertext.
7. Verify the original message.

Populate:

Message	p	g	Private x	Public y	k	Ciphertext	Decrypted Message

Part C – Diffie–Hellman

Generate the common parameters p, g.

Calculate:

- Alice's private key
- Bob's private key
- Alice's public value
- Bob's public value
- Alice's shared secret
- Bob's shared secret

Populate:

p	g	Alice Private	Alice Public	Bob Private	Bob Public	Alice Secret	Bob Secret	Match?

Part D – ECC

Generate:

- Curve parameters
- Base point G
- Private key
- Public key

Perform the required ECC operations and record the results.

Curve	G	Private Key	Public Key	Result

7. Experimental Analysis

The main objective is to understand **why different asymmetric algorithms are designed differently**.

Part A – Mathematical Foundation

Complete the following table.

Property	RSA	ElGamal	Diffie–Hellman	ECC
Mathematical Basis				
Large Prime Required				
Modular Arithmetic				
Elliptic Curve Operations				
Main Hard Problem				

Part B – Security Assumption

Complete:

Algorithm	Underlying Hard Problem
RSA	Integer Factorization
ElGamal	Discrete Logarithm
Diffie–Hellman	Discrete Logarithm / Diffie–Hellman Problem
ECC	Elliptic Curve Discrete Logarithm

Students should explain in their own words **why solving the underlying mathematical problem would compromise the security of the algorithm**.

8. Key Size Analysis

Compare the approximate key sizes required to achieve equivalent security.

Security Level	RSA	ECC
~128-bit		
~192-bit		
~256-bit		

Question

Why can ECC provide comparable security using significantly smaller keys than RSA?

9. Key Management Analysis

Property	RSA	ElGamal	Diffie–Hellman	ECC
Public Key				
Private Key				
Encryption				
Decryption				
Key Exchange				
Shared Secret				
Requires Public Parameters				

10. Encryption vs Key Exchange

Classify the algorithms according to their primary purpose.

Algorithm	Encryption	Key Exchange
RSA		
ElGamal		
Diffie–Hellman		
ECC		

Students must explain:

Why does Diffie–Hellman establish a shared secret but does not itself encrypt the message?

11. Comparative Evaluation

Rate each algorithm from **1 (low) to 5 (high)**.

Parameter	RSA	ElGamal	DH	ECC
Key Generation Complexity				
Mathematical Complexity				
Key Size Efficiency				
Encryption Capability				
Key Exchange Capability				
Scalability				
Suitability for Resource-Constrained Devices				
Overall Practical Suitability				

Students must **justify the ratings** rather than simply assigning numbers.

12. Discussion Questions

1. Why does RSA require two prime numbers during key generation?
2. Why must e and $\phi(n)$ be relatively prime in RSA?
3. Why is prime generation important in RSA and ElGamal?
4. What mathematical problem forms the basis of RSA security?
5. What mathematical problem forms the basis of ElGamal and Diffie–Hellman security?
6. Why does Diffie–Hellman establish a shared secret without directly transmitting the secret?
7. Why does Diffie–Hellman not provide message encryption by itself?
8. Why can ECC provide comparable security with smaller key sizes?
9. Which algorithm would you prefer for a resource-constrained device? Justify your answer.
10. Why do modern cryptographic systems often combine asymmetric key exchange with symmetric encryption?

Submission:

- Python source code
- Completed Excel workbook
- Document file with answers to the discussion questions and Graphs:
    - RSA vs ECC Key Size Comparison (Line Graph)
    - Algorithm Capability Comparison (Radar Chart)
- A one-page conclusion summarizing the evolution of classical symmetric ciphers and their relationship to modern symmetric encryption.